

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Lab Organización y Arquitectura de Computadoras
Grupo: 03

Práctica 3

P R E S E N T A
Rea Alberto Cristian | 318273130
Ortega de la Paz Rafael | 420054085

PROFESOR DE LABORATORIO
Ricardo Daniel Vega Gutierrez

Fecha de entrega: 13 de Octubre de 2026
Semestre 2027-1



Objetivo

Diseñar una máquina de estados de cinco estados, tres entradas y cuatro salidas a partir de su carta ASM, obtener sus ecuaciones con mapas de Karnaugh e implementarla con flip-flops tipo D en la DE10-Lite, comprobando que el comportamiento de la tarjeta coincide con el análisis teórico.

Introducción

En esta práctica diseñamos y armamos una máquina de estados con flip-flops tipo D y la implementamos en la tarjeta DE10-Lite. En la Práctica 1 solo conectamos un contador que ya existía (el 74193); ahora el camino completo fue nuestro: carta ASM, codificación de los estados, tabla de transiciones, mapas de Karnaugh para cada flip-flop y cada salida, y por último el código en VHDL.

Para que la carta tuviera sentido escogimos un caso real, el control de un portón eléctrico de cochera. El botón de mando (A) abre y cierra, un sensor detecta obstáculos (B) y un fin de carrera (C) avisa cuando el portón terminó de moverse. La máquina maneja el motor de apertura, el motor de cierre, una luz de cortesía y una alarma. Cada estado solo revisa las entradas que están en su trayectoria, que es la idea del direccionamiento por trayectoria que pide la práctica.

Desarrollo

Convención de las señales

Antes de dibujar la carta fijamos qué significa cada señal, porque la práctica pide usar la misma convención en la carta, la tabla, los mapas y el VHDL. Las tres entradas son interruptores de la tarjeta. Z1 y Z2 usan lógica estándar (1 = activa). Z3 y Z4 usan lógica negada: están activas cuando valen 0 y se apagan con 1. Los displays de siete segmentos de la DE10-Lite encienden cada segmento con un 0, así que mandamos las dos salidas negadas a segmentos del HEX0; de esa forma el segmento prende justo cuando la salida está activa, igual que los LED de las salidas estándar.

Señal	Tipo	Función	Convención	En la tarjeta
A	Entrada	Botón de mando (abrir / cerrar)	1 = presionado	SW0
B	Entrada	Sensor de obstáculo	1 = hay obstáculo	SW1
C	Entrada	Fin de carrera del movimiento en curso	1 = llegó al final	SW2
Z1	Salida estándar	Motor de apertura	1 = activa, 0 = inactiva	LEDR0
Z2	Salida estándar	Luz de cortesía	1 = activa, 0 = inactiva	LEDR1
Z3	Salida negada	Motor de cierre	0 = activa, 1 = inactiva	HEX0, segmento a
Z4	Salida negada	Alarma	0 = activa, 1 = inactiva	HEX0, segmento d

Tabla 1. Señales de la máquina y convención de activación.

Como queremos ver cada transición, el reloj no es el de 50 MHz sino el botón KEY0: un pulso del botón es un ciclo de reloj. El reset asíncrono es KEY1 y deja la máquina en E0.

Carta ASM

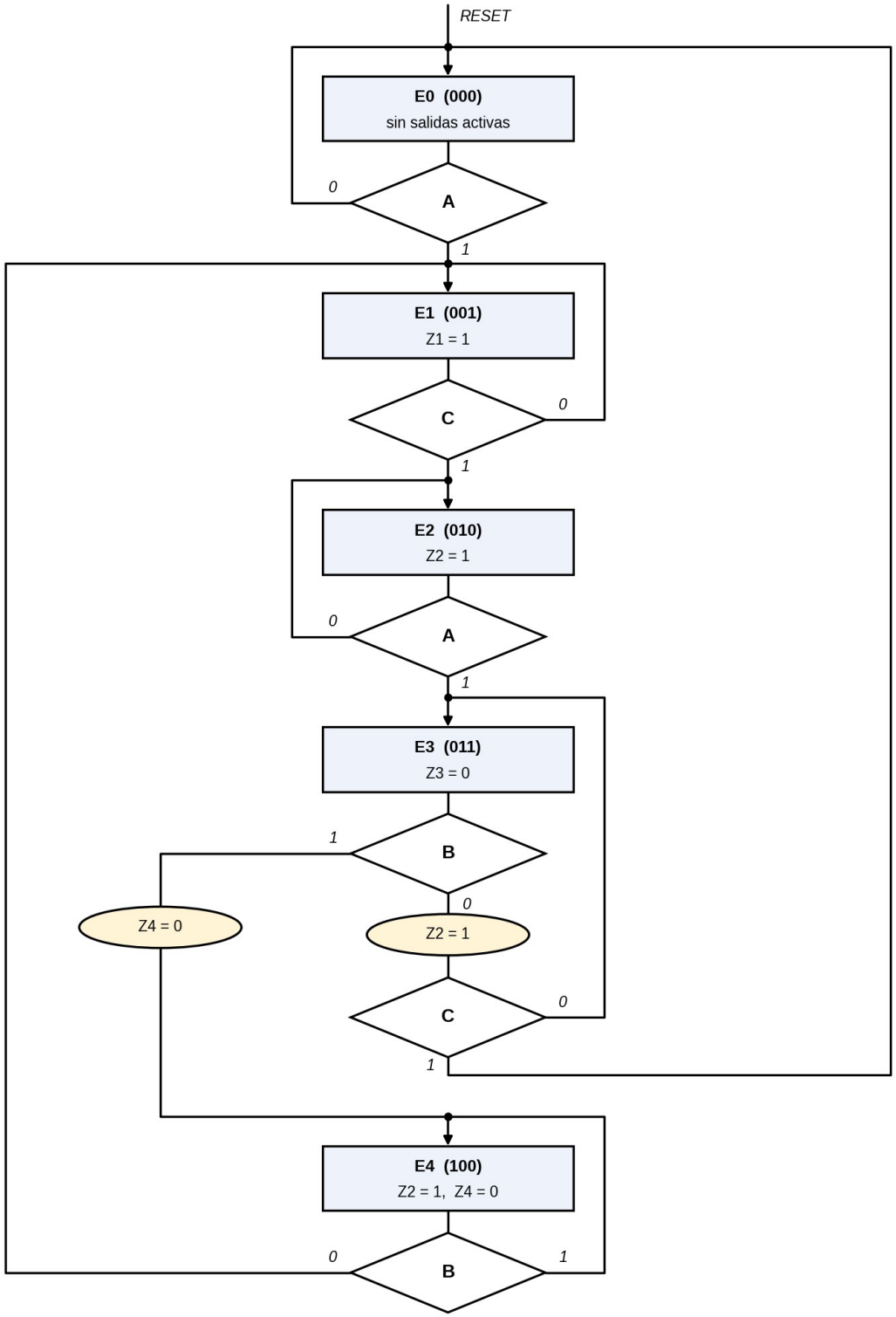
La máquina arranca en E0 (portón cerrado). Al presionar A pasa a E1 y abre el portón hasta que C avisa que llegó al final (E2, portón abierto, con la luz encendida). Con otro A pasa a E3 y cierra. Si mientras cierra aparece un obstáculo (B = 1) se va a E4: se detiene el motor, quedan

encendidas la luz y la alarma, y cuando el obstáculo se quita (B = 0) el portón vuelve a abrirse (E1) en lugar de seguir cerrando. Si no hay obstáculo y C llega a 1, termina en E0.

Estado	Q2Q1Q0	Salidas del estado	Revisa	Siguiente estado
E0	000	Ninguna activa	A	A = 0 → E0; A = 1 → E1
E1	001	Z1 = 1	C	C = 0 → E1; C = 1 → E2
E2	010	Z2 = 1	A	A = 0 → E2; A = 1 → E3
E3	011	Z3 = 0	B, luego C	B = 1 (Z4 = 0) → E4; B = 0 (Z2 = 1) y C = 1 → E0; B = 0 (Z2 = 1) y C = 0 → E3
E4	100	Z2 = 1, Z4 = 0	B	B = 1 → E4; B = 0 → E1

Tabla 2. Estados, codificación, salidas y trayectorias de la carta ASM.

El estado E3 tiene dos salidas condicionales: por la rama B = 1 sale Z4 = 0 (alarma) y por la rama B = 0 sale Z2 = 1 (luz). Las dos ramas de B llevan salida, que es lo que pide el punto 4 de los requisitos. Por eso Z2 y Z4 no dependen solo del estado, también dependen de B. En total son cinco estados, tres entradas y cuatro salidas (dos estándar y dos negadas).



Z1, Z2: lógica estándar (1 = activa) Z3, Z4: lógica negada (0 = activa)
Entradas: A, B, C Los óvalos son salidas condicionales

Figura 1. Carta ASM de la máquina (convención: Z1 y Z2 activas en 1; Z3 y Z4 activas en 0).

Codificación de los estados

Con cinco estados hacen falta $\lceil \log_2 5 \rceil = 3$ bits, o sea tres flip-flops (Q2, Q1, Q0). Usamos código binario consecutivo, en el orden en que numeramos los estados (E0 = 000, E1 = 001, E2 = 010, E3 = 011 y E4 = 100, como se ve en la Tabla 2). Sobran tres combinaciones (101, 110 y 111) que no son estados de la máquina; en los mapas las tratamos como condiciones que no importan.

Tabla de transiciones

Con tres bits de estado y tres entradas hay 64 combinaciones. Las 40 de la Tabla 3 son las que corresponden a los cinco estados válidos; las otras 24 (estado actual 101, 110 o 111) no aparecen porque son X en todas las funciones. Las entradas que no están en la trayectoria de un estado se escribieron completas de todos modos, así la tabla tiene las 8 combinaciones por estado y se puede pasar directo a los mapas. Z3 y Z4 se anotan con su nivel real (0 = activa). D2, D1 y D0 son las entradas de los flip-flops, es decir, el estado siguiente.

Estado actual				Entradas			Estado siguiente				Salidas			
Edo.	**Q2**	**Q1**	**Q0**	**A**	**B**	**C**	**D2**	**D1**	**D0**	**Edo.**	**Z1**	**Z2**	**Z3**	**Z4**
E0	0	0	0	0	0	0	0	0	0	**E0**	0	0	1	1
E0	0	0	0	0	0	1	0	0	0	**E0**	0	0	1	1
E0	0	0	0	0	1	0	0	0	0	**E0**	0	0	1	1
E0	0	0	0	0	1	1	0	0	0	**E0**	0	0	1	1
E0	0	0	0	1	0	0	0	0	1	**E1**	0	0	1	1
E0	0	0	0	1	0	1	0	0	1	**E1**	0	0	1	1
E0	0	0	0	1	1	0	0	0	1	**E1**	0	0	1	1
E0	0	0	0	1	1	1	0	0	1	**E1**	0	0	1	1
E1	0	0	1	0	0	0	0	0	1	**E1**	1	0	1	1
E1	0	0	1	0	0	1	0	1	0	**E2**	1	0	1	1
E1	0	0	1	0	1	0	0	0	1	**E1**	1	0	1	1
E1	0	0	1	0	1	1	0	1	0	**E2**	1	0	1	1
E1	0	0	1	1	0	0	0	0	1	**E1**	1	0	1	1
E1	0	0	1	1	0	1	0	1	0	**E2**	1	0	1	1
E1	0	0	1	1	1	0	0	0	1	**E1**	1	0	1	1
E1	0	0	1	1	1	1	0	1	0	**E2**	1	0	1	1
E2	0	1	0	0	0	0	0	1	0	**E2**	0	1	1	1
E2	0	1	0	0	0	1	0	1	0	**E2**	0	1	1	1
E2	0	1	0	0	1	0	0	1	0	**E2**	0	1	1	1
E2	0	1	0	0	1	1	0	1	0	**E2**	0	1	1	1
E2	0	1	0	1	0	0	0	1	1	**E3**	0	1	1	1
E2	0	1	0	1	0	1	0	1	1	**E3**	0	1	1	1
E2	0	1	0	1	1	0	0	1	1	**E3**	0	1	1	1
E2	0	1	0	1	1	1	0	1	1	**E3**	0	1	1	1
E3	0	1	1	0	0	0	0	1	1	**E3**	0	1	0	1
E3	0	1	1	0	0	1	0	0	0	**E0**	0	1	0	1
E3	0	1	1	0	1	0	1	0	0	**E4**	0	0	0	0
E3	0	1	1	0	1	1	1	0	0	**E4**	0	0	0	0
E3	0	1	1	1	0	0	0	1	1	**E3**	0	1	0	1
E3	0	1	1	1	0	1	0	0	0	**E0**	0	1	0	1
E3	0	1	1	1	1	0	1	0	0	**E4**	0	0	0	0
E3	0	1	1	1	1	1	1	0	0	**E4**	0	0	0	0
E4	1	0	0	0	0	0	0	0	1	**E1**	0	1	1	0
E4	1	0	0	0	0	1	0	0	1	**E1**	0	1	1	0
E4	1	0	0	0	1	0	1	0	0	**E4**	0	1	1	0
E4	1	0	0	0	1	1	1	0	0	**E4**	0	1	1	0
E4	1	0	0	1	0	0	0	0	1	**E1**	0	1	1	0
E4	1	0	0	1	0	1	0	0	1	**E1**	0	1	1	0
E4	1	0	0	1	1	0	1	0	0	**E4**	0	1	1	0
E4	1	0	0	1	1	1	1	0	0	**E4**	0	1	1	0

Tabla 3. Tabla de transiciones completa (estado actual, entradas, estado siguiente y salidas).

Mapas de Karnaugh

Cada función depende de seis variables (Q2, Q1, Q0, A, B y C), así que cada mapa es de 8 × 8: los renglones son Q2Q1Q0 y las columnas ABC, los dos en código Gray para que las celdas vecinas cambien un solo bit. Los bordes también son vecinos (el renglón 000 con el 100 y la columna 000 con la 100), por eso algunos grupos aparecen partidos. Las X son las 24 combinaciones de los códigos que no usamos y las aprovechamos cuando ayudan a agrandar un grupo. Cada grupo tiene un color y ese color es el del término correspondiente debajo del mapa. En las ecuaciones el apóstrofo indica negación (Q1' es Q1 negada) y el punto es AND.

Mapa de D2

Mapa de Karnaugh de D2 (entrada del flip-flop Q2)

Q2 Q1 Q0 \ A B C	000	001	011	010	110	111	101	100
000	0	0	0	0	0	0	0	0
001	0	0	0	0	0	0	0	0
011	0	0	**1**	**1**	**1**	**1**	0	0
010	0	0	0	0	0	0	0	0
110	X	X	X	X	X	X	X	X
111	X	X	X	X	X	X	X	X
101	X	X	X	X	X	X	X	X
100	0	0	**1**	**1**	**1**	**1**	0	0

D2 = Q1·Q0·B + Q2·B

Grupos: Q1·Q0·B Q2·B

X = condición no importa (códigos 101, 110 y 111 no usados)

Figura 2. Mapa de Karnaugh de D2 con sus grupos y su ecuación.

D2 solo vale 1 cuando el estado siguiente es E4 (100). Eso pasa en dos casos: estando en E3 con B = 1 (término Q1·Q0·B) o estando ya en E4 con B = 1 (término Q2·B). Como 101, 110 y 111 no se usan, para el segundo grupo basta con Q2·B.

Mapa de D1

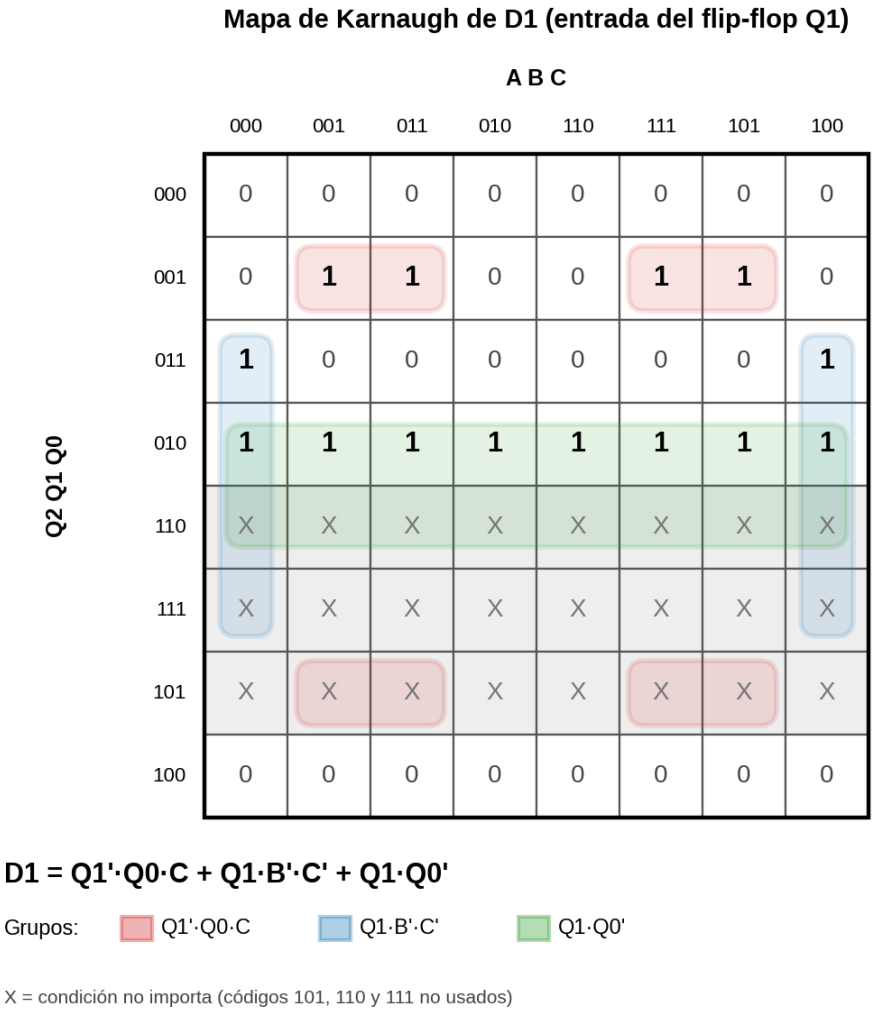


Figura 3. Mapa de Karnaugh de D1 con sus grupos y su ecuación.

D1 vale 1 cuando el siguiente estado es E2 o E3 (010 y 011). El término Q1'·Q0·C es el salto de E1 a E2. Q1·Q0' cubre a E2, que sin importar A termina en E2 o en E3. Q1·B'·C' es E3 quedándose en E3 (B = 0 y C = 0).

Mapa de D0

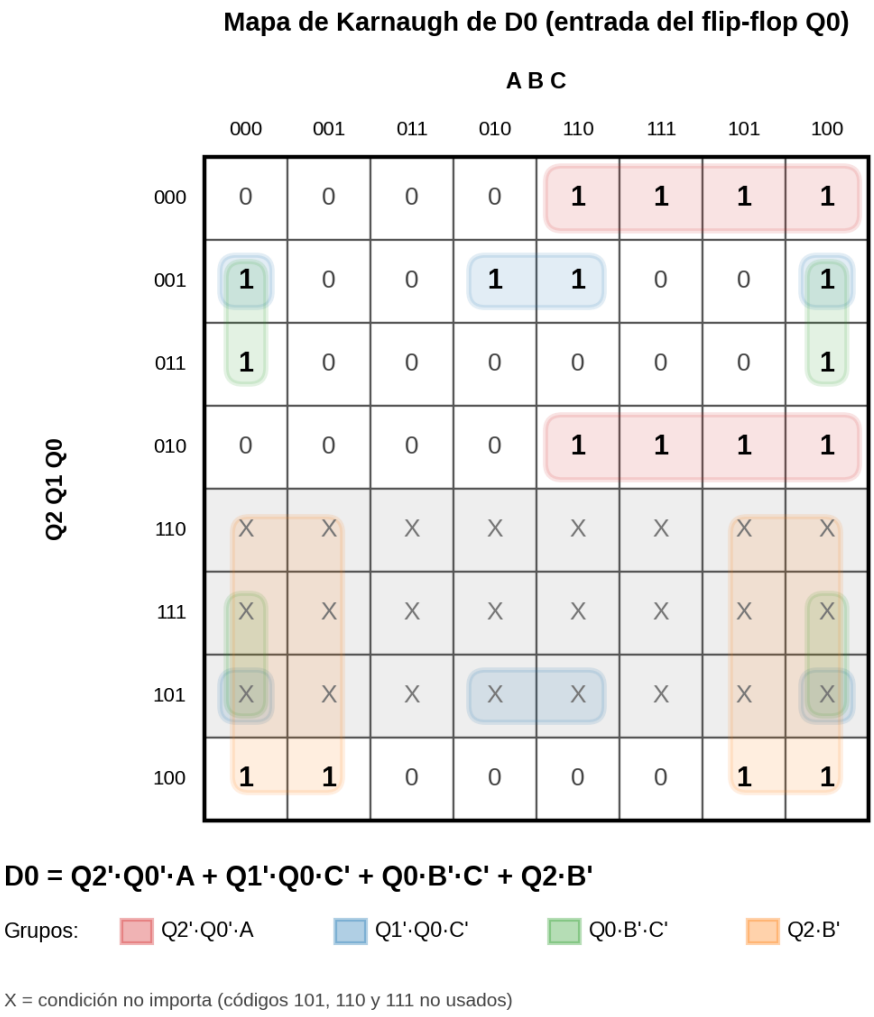


Figura 4. Mapa de Karnaugh de D0 con sus grupos y su ecuación.

D0 vale 1 cuando el siguiente estado es E1 o E3 (001 y 011). Q2'·Q0'·A cubre E0 → E1 y E2 → E3, que son los dos con A = 1. Q1'·Q0·C' es E1 quedándose en E1. Q0·B'·C' es E3 quedándose en E3 (se traslapa con el término anterior, pero así el grupo es más grande). Q2·B' es E4 regresando a E1 cuando B = 0.

Mapa de Z1

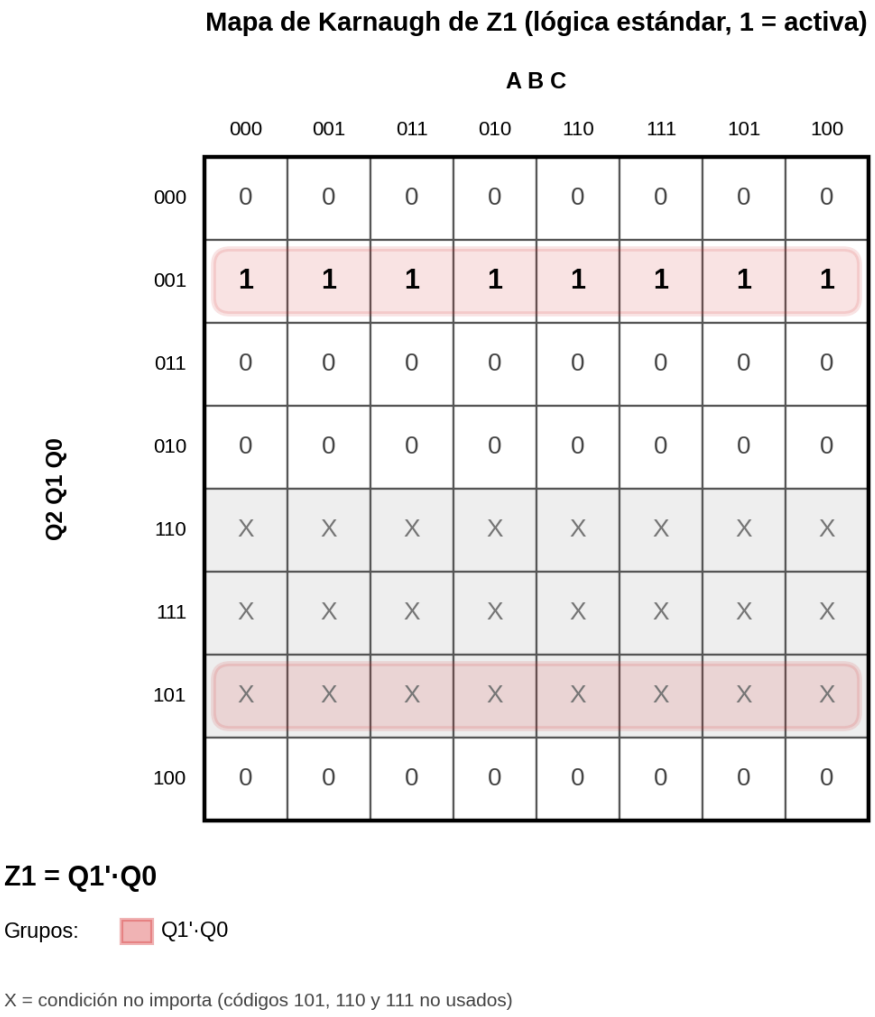


Figura 5. Mapa de Karnaugh de Z1 con su grupo y su ecuación.

Z1 solo está activa en E1 (001). Con la X del código 101 el término se reduce a Q1'·Q0.

Mapa de Z2

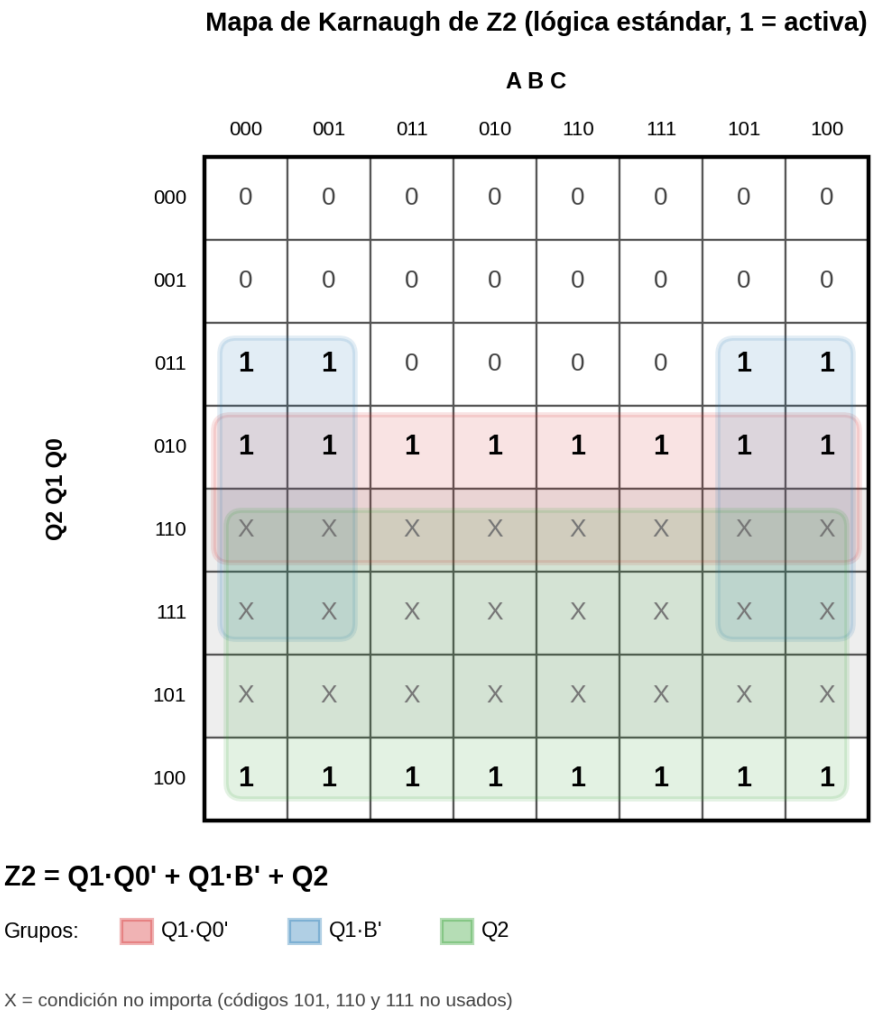


Figura 6. Mapa de Karnaugh de Z2 con sus grupos y su ecuación.

Z2 vale 1 en E2 (Q1·Q0'), en E4 (Q2) y en E3 cuando B = 0 (Q1·B'). Esta última es la salida condicional de la rama B = 0 de la carta.

Mapa de Z3

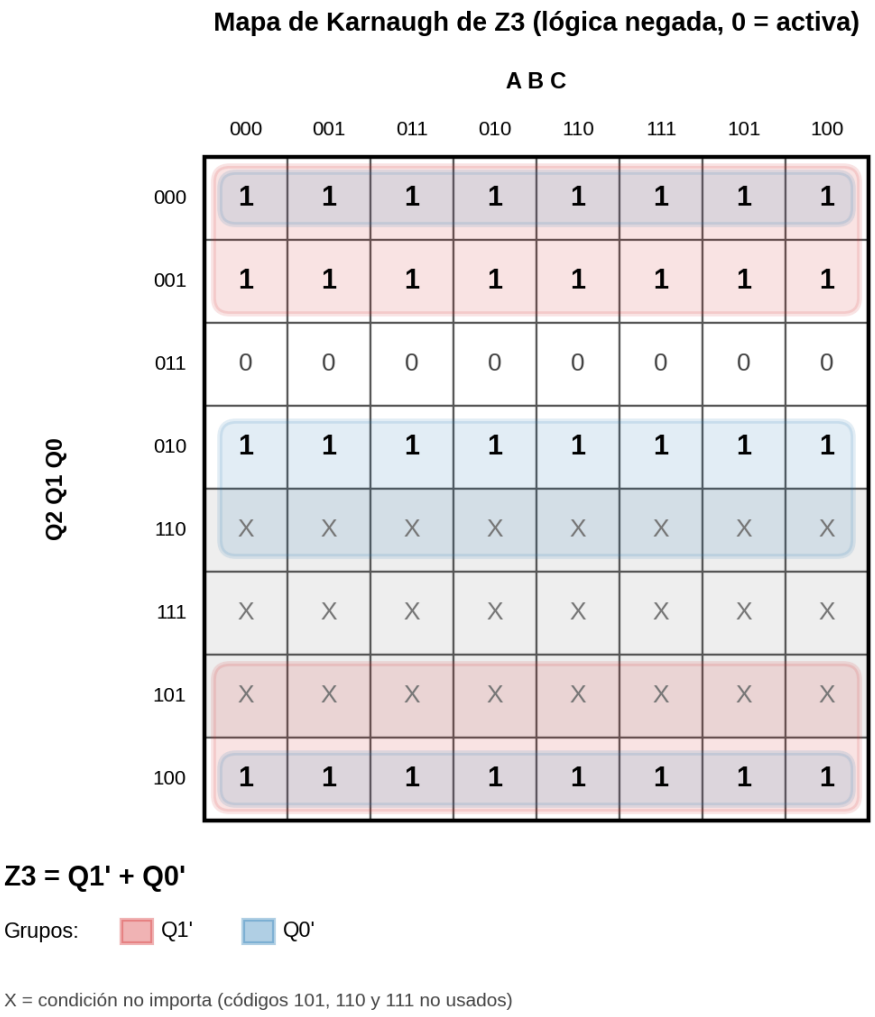


Figura 7. Mapa de Karnaugh de Z3 (lógica negada) con sus grupos y su ecuación.

Z3 es la salida negada del motor de cierre, así que en el mapa se anota su nivel real: vale 0 solo en E3 y 1 en todos los demás estados. Los unos forman Z3 = Q1' + Q0'. Dicho de otra forma, Z3 se activa (se pone en 0) únicamente cuando Q1·Q0 = 1.

Mapa de Z4

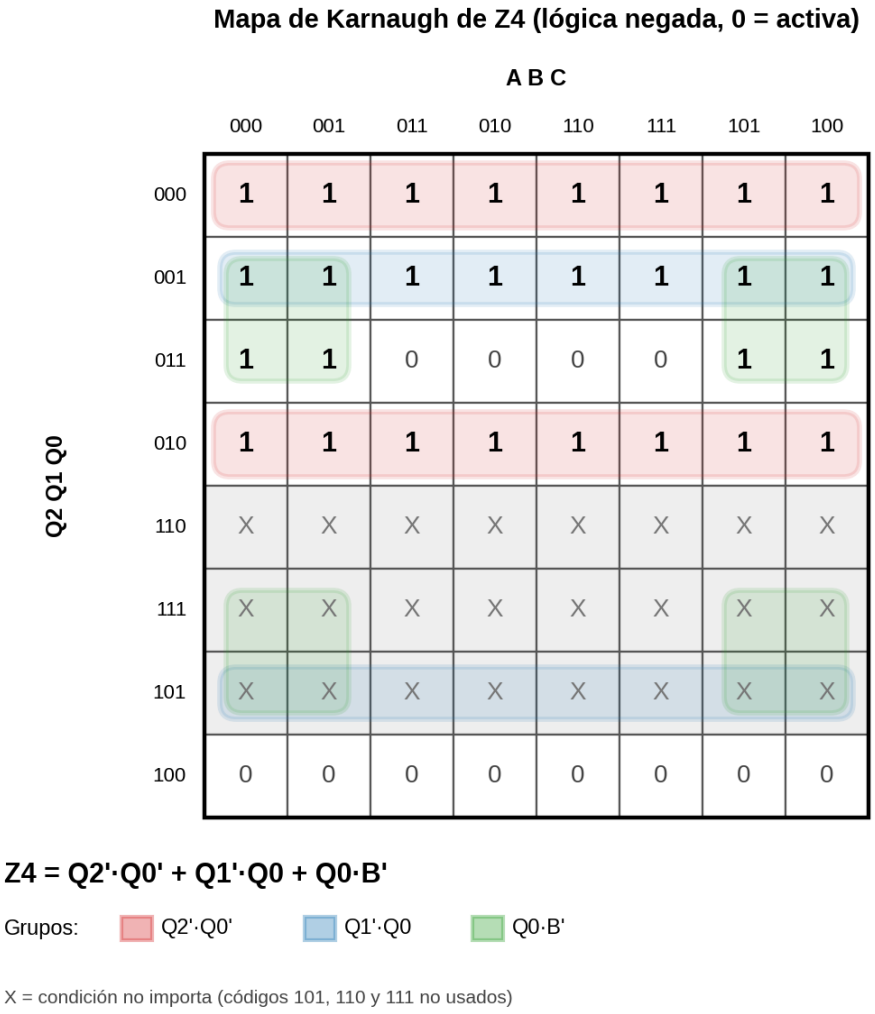


Figura 8. Mapa de Karnaugh de Z4 (lógica negada) con sus grupos y su ecuación.

Z4 es la alarma negada: vale 0 en E4 y en E3 con B = 1. También se mapeó el nivel real, así que los unos están en E0 y E2 (Q2'·Q0'), en E1 (Q1'·Q0) y en E3 con B = 0 (Q0·B').

En la Tabla 4 se juntan las siete ecuaciones simplificadas. Las comprobamos contra las 40 filas de la Tabla 3 y coinciden en todas.

Función	Ecuación simplificada
D2	Q1·Q0·B + Q2·B
D1	Q1'·Q0·C + Q1·B'·C' + Q1·Q0'
D0	Q2'·Q0'·A + Q1'·Q0·C' + Q0·B'·C' + Q2·B'
Z1	Q1'·Q0
Z2	Q1·Q0' + Q1·B' + Q2
Z3	Q1' + Q0'
Z4	Q2'·Q0' + Q1'·Q0 + Q0·B'

Tabla 4. Ecuaciones simplificadas de los flip-flops y de las salidas.

Circuito con flip-flops tipo D

El circuito de la Figura 9 se dibujó directamente con las ecuaciones de la Tabla 4. A la izquierda están las entradas y las salidas de los flip-flops con sus complementos; cada punto negro indica qué variable entra a cada compuerta AND. Cada AND es un término de la suma de productos y cada OR junta los términos de una función. Las tres primeras OR alimentan las entradas D2, D1 y D0 de los flip-flops, que comparten el reloj manual (KEY0, flanco de bajada) y el reset asíncrono (KEY1). Las salidas Q y Q' de los flip-flops regresan a las líneas verticales de la izquierda, lo que cierra el lazo de la máquina de estados. Las compuertas de abajo forman la lógica de salidas; Z2 y Z4 reciben B además del estado, por eso son las salidas condicionales.

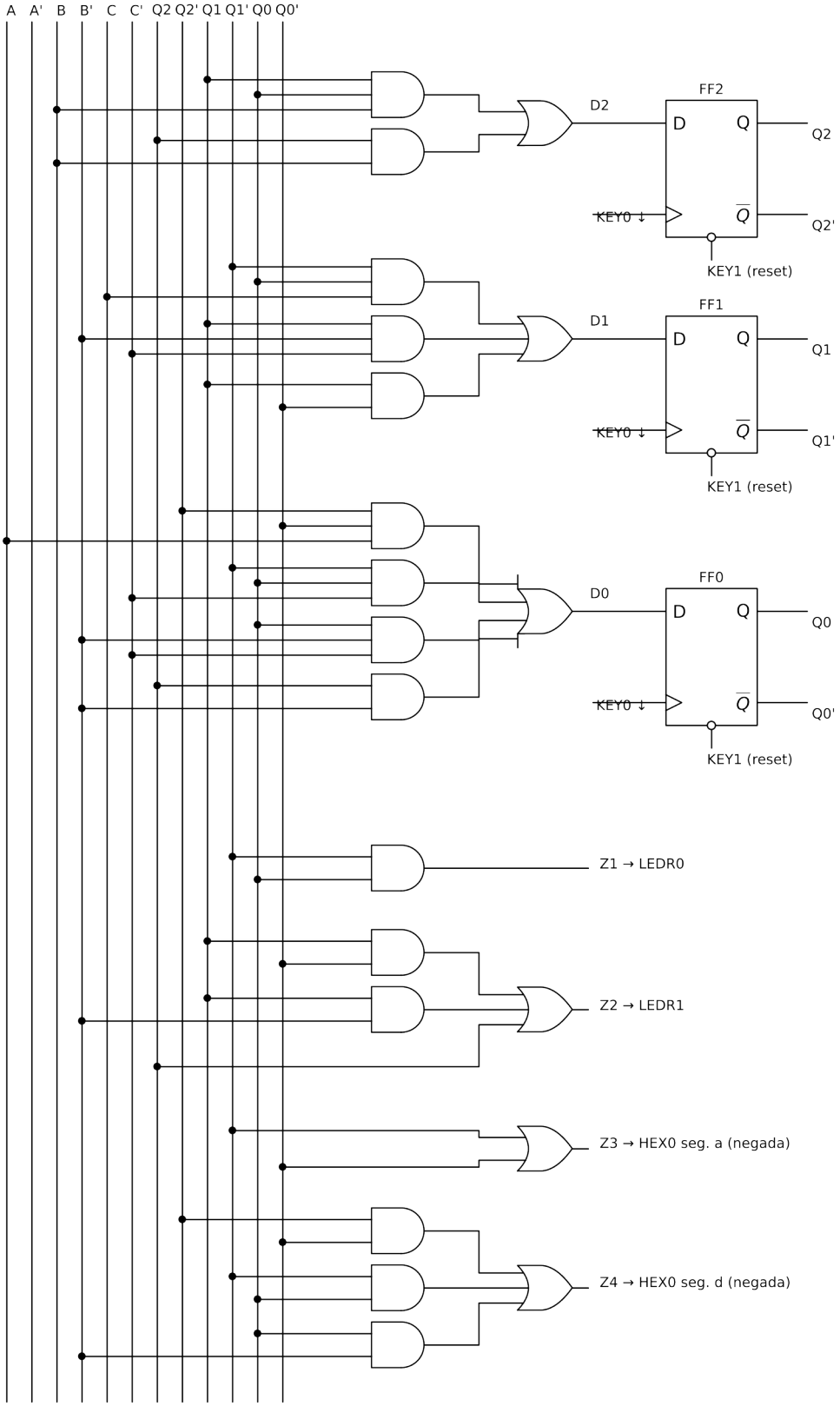


Figura 9. Circuito con tres flip-flops tipo D, lógica de estado siguiente y lógica de salidas.

Estados no usados

Los códigos 101, 110 y 111 nunca se alcanzan partiendo de E0, porque todas las transiciones de la Tabla 3 llevan a E0–E4. Aun así revisamos qué harían las ecuaciones si el circuito arrancara en uno de ellos. Desde 111 el siguiente estado siempre es E1, E3 o E4. Desde 101 y 110 depende de las entradas: puede pasar a un estado válido (E1 o E3) o quedarse en un código no usado (101 puede ir a 101 o 110, y 110 puede quedarse en 110). Como en la tarjeta siempre empezamos con KEY1 (reset), que fuerza el estado 000, no agregamos lógica extra para recuperarse de esos códigos.

Implementación en VHDL

El código se escribió siguiendo el análisis: las ecuaciones de los mapas pasan casi directo a asignaciones concurrentes y solo los flip-flops van dentro de un proceso. A continuación se explican los bloques principales.

Entidad y puertos. Los puertos coinciden con lo que hay en la tarjeta: KEY[1:0] (KEY0 es el reloj y KEY1 el reset), SW[2:0] para A, B y C, LEDR[1:0] para Z1 y Z2, y HEX0[6:0] para los siete segmentos del primer display, donde irán Z3 y Z4.

```
entity practica3 is
  port (
    KEY  : in  std_logic_vector(1 downto 0);
    SW   : in  std_logic_vector(2 downto 0);
    LEDR : out std_logic_vector(1 downto 0);
    HEX0 : out std_logic_vector(6 downto 0)
  );
end practica3;
```

Estado siguiente. Cada D es la ecuación de su mapa tal cual (Tabla 4). Elegimos asignaciones concurrentes y no un case por estado porque así el código se puede comparar renglón por renglón con los mapas de Karnaugh.

```
d2 <= (q1 and q0 and b) or (q2 and b);
d1 <= ((not q1) and q0 and c) or (q1 and (not b) and (not c))
      or (q1 and (not q0));
d0 <= ((not q2) and (not q0) and a) or ((not q1) and q0 and (not c))
      or (q0 and (not b) and (not c)) or (q2 and (not b));
```

Flip-flops tipo D. Un solo proceso guarda q2, q1 y q0. El reset es asíncrono y activo en bajo (KEY1), por eso se evalúa antes que el reloj. Los botones de la tarjeta valen 1 sin presionar y 0 al presionar, así que el flanco de bajada de KEY0 ocurre justo cuando se aprieta el botón. En cada flanco, q toma el valor de su d.

```
process (clk, rst_n)
begin
  if rst_n = '0' then
    q2 <= '0';  q1 <= '0';  q0 <= '0';
  elsif falling_edge(clk) then
    q2 <= d2;  q1 <= d1;  q0 <= d0;
  end if;
end process;
```

Salidas. z1 a z4 son las ecuaciones de los mapas. Z3 y Z4 se dejan con su nivel real (0 = activa), igual que en la tabla. La última línea arma el vector de los segmentos en el orden g f e d c b a: Z4 queda en el bit 3 (segmento d), Z3 en el bit 0 (segmento a) y el resto en 1 para que esos segmentos permanezcan apagados.

```
z1 <= (not q1) and q0;
z2 <= (q1 and (not q0)) or (q1 and (not b)) or q2;
z3 <= (not q1) or (not q0);
z4 <= ((not q2) and (not q0)) or ((not q1) and q0) or (q0 and (not b));

LEDR(0) <= z1;
LEDR(1) <= z2;
HEX0    <= "111" & z4 & "11" & z3;
```

Asignación de pines

Las entradas y salidas se asignaron a los pines de la DE10-Lite que indica el manual de la tarjeta (Tabla 5).

Puerto	Pin	Tipo	Uso
KEY[0]	PIN_B8	Entrada	Reloj manual (flanco de bajada al presionar)
KEY[1]	PIN_A7	Entrada	Reset asíncrono, activo en bajo
SW[0]	PIN_C10	Entrada	A: botón de mando
SW[1]	PIN_C11	Entrada	B: sensor de obstáculo
SW[2]	PIN_D12	Entrada	C: fin de carrera
LEDR[0]	PIN_A8	Salida	Z1: motor de apertura
LEDR[1]	PIN_A9	Salida	Z2: luz de cortesía
HEX0[0]	PIN_C14	Salida	Segmento a: Z3 (motor de cierre, negada)
HEX0[3]	PIN_C16	Salida	Segmento d: Z4 (alarma, negada)
HEX0[1], [2], [4], [5], [6]	E15, C15, E16, D17, C17	Salida	Segmentos b, c, e, f, g: fijos en 1 (apagados)

Tabla 5. Asignación de pines.

Compilación, simulación y pruebas en la tarjeta

Compilación. Se creó el proyecto practica3 en Quartus para el dispositivo MAX 10 10M50DAF484C7G de la DE10-Lite, con practica3.vhd como entidad principal, y se compiló con Start Compilation.

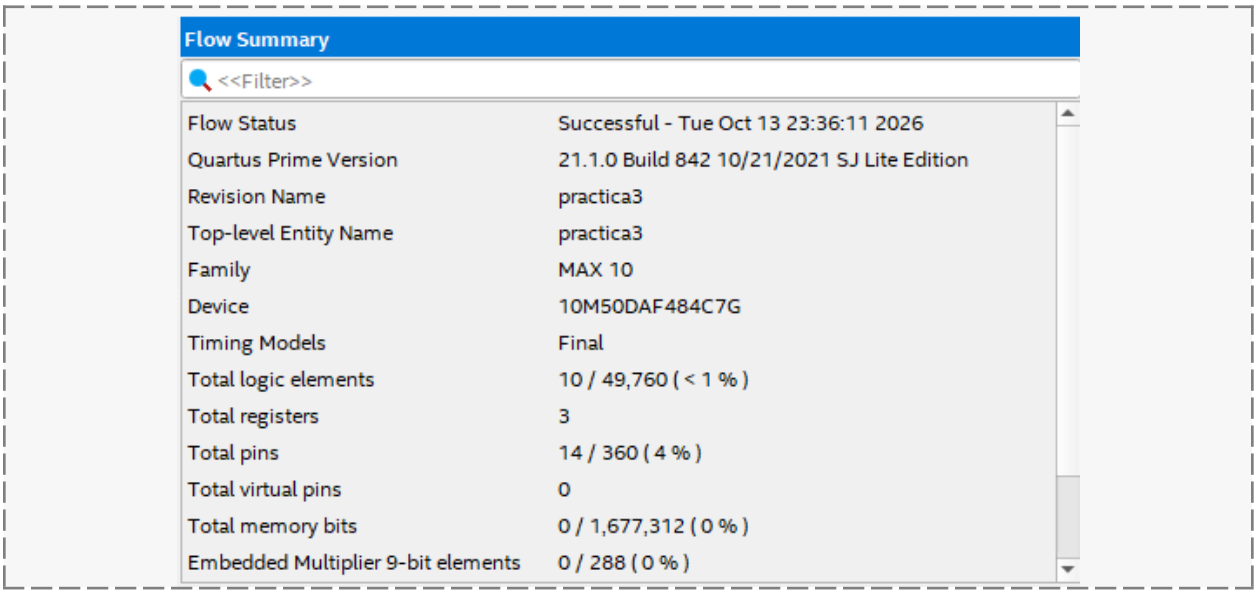


Figura 10. Resultado de la compilación en Quartus.

Simulación. La simulación funcional se hizo con un banco de pruebas en VHDL (tb.vhd) que instancia a practica3, aplica primero el reset con KEY1 y después recorre la secuencia de la Tabla 6, dando un pulso de KEY0 por paso. Antes de cada pulso el banco compara Z1–Z4 contra los valores esperados de la tabla con una instrucción assert. La secuencia recorre una vez cada una de las 11 flechas de la carta ASM (cuatro lazos propios y siete cambios de estado) y la simulación terminó sin ninguna discrepancia. Las formas de onda se muestran en la Figura 11.

Paso	Entradas			Estado		Salidas				Transición que se prueba
	A	B	C	Actual	Siguiente	Z1	Z2	Z3	Z4	
1	0	0	0	E0	E0	0	0	1	1	E0 se queda (A = 0)
2	1	0	0	E0	E1	0	0	1	1	E0 → E1 (A = 1)
3	0	0	0	E1	E1	1	0	1	1	E1 se queda (C = 0)
4	0	0	1	E1	E2	1	0	1	1	E1 → E2 (C = 1)
5	0	0	0	E2	E2	0	1	1	1	E2 se queda (A = 0)
6	1	0	0	E2	E3	0	1	1	1	E2 → E3 (A = 1)
7	0	0	0	E3	E3	0	1	0	1	E3 se queda (B = 0, C = 0)
8	0	1	0	E3	E4	0	0	0	0	E3 → E4 (B = 1), suena la alarma
9	0	1	0	E4	E4	0	1	1	0	E4 se queda (B = 1)
10	0	0	0	E4	E1	0	1	1	0	E4 → E1 (B = 0)
11	0	0	1	E1	E2	1	0	1	1	E1 → E2 otra vez (C = 1)
12	1	0	0	E2	E3	0	1	1	1	E2 → E3 otra vez (A = 1)
13	0	0	1	E3	E0	0	1	0	1	E3 → E0 (B = 0, C = 1)

Tabla 6. Secuencia de prueba y resultado esperado (salidas calculadas con el estado actual y las entradas del paso).

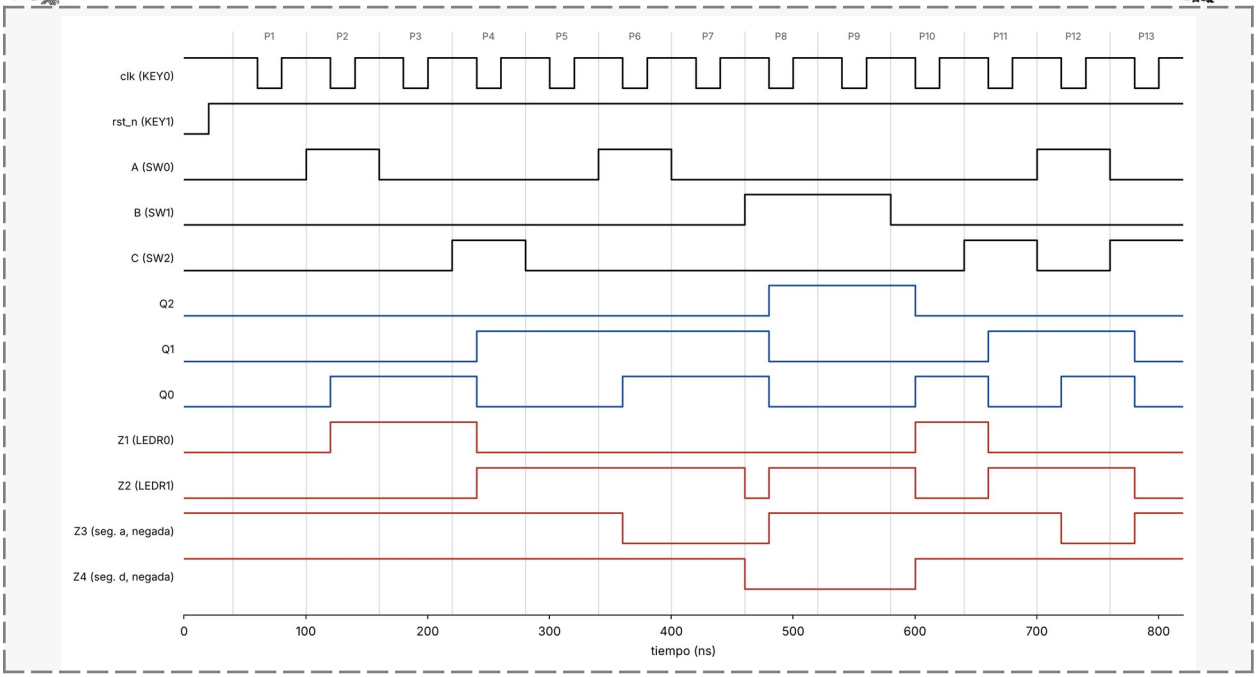


Figura 11. Simulación funcional de la secuencia de prueba (P1 a P13 corresponden a los pasos de la Tabla 6).

Pruebas en la tarjeta. Con la compilación terminada se abrió el Programmer, se seleccionó el cable USB-Blaster en modo JTAG y se cargó output_files/practica3.sof en el 10M50DAF484; la programación terminó en 100 % (Successful), como se ve en la Figura 12. Con la tarjeta ya programada se repitió en la DE10-Lite la misma secuencia de la Tabla 6: reset con KEY1, entradas con SW0, SW1 y SW2, y un pulso de KEY0 por cada paso. En todos los pasos los LED LEDR0 y LEDR1 y los segmentos a y d del HEX0 mostraron el estado siguiente y las salidas esperadas.

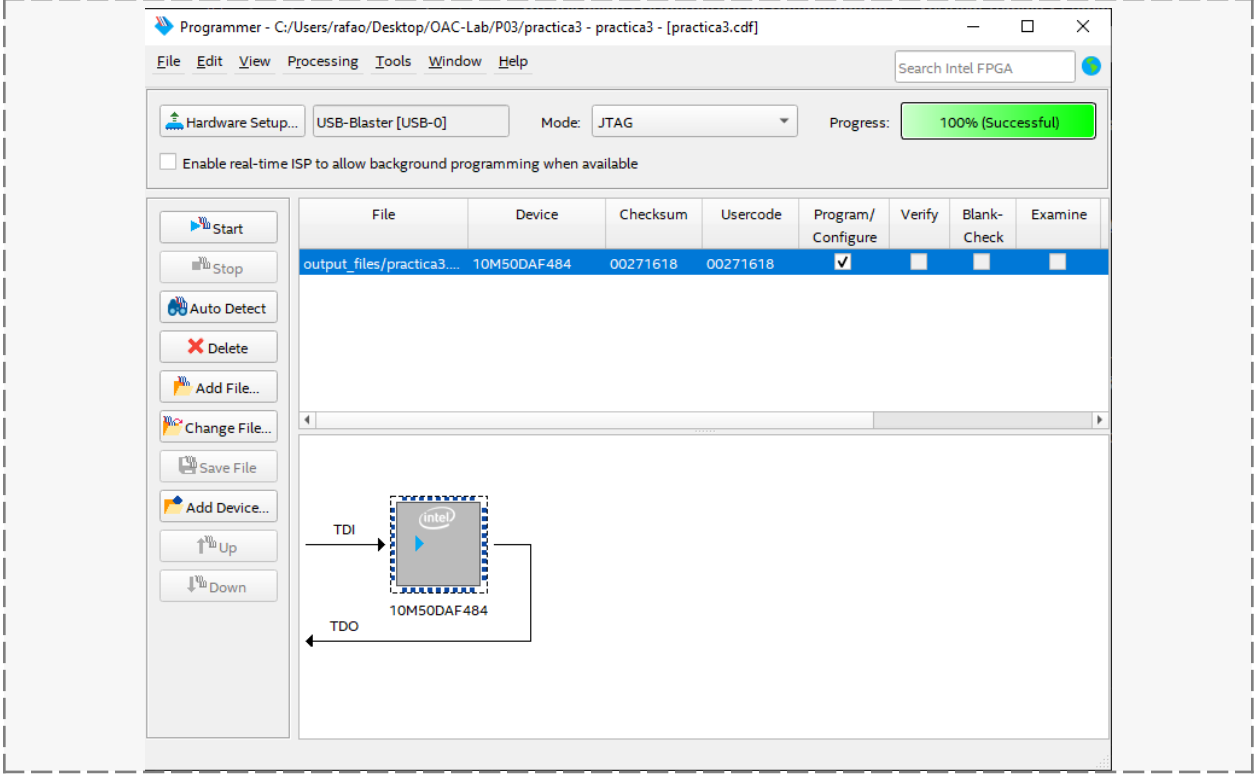


Figura 12. Programación de la DE10-Lite con el archivo practica3.sof.

Comparación con el análisis teórico. Tanto en la simulación como en la tarjeta, los 13 pasos de la Tabla 6 dieron el estado siguiente y las salidas esperadas, así que las ecuaciones de la Tabla 4 y el código VHDL reproducen la carta ASM. Z3 y Z4, al ser negadas, valen 0 justo donde la carta las activa: Z3 solo en E3 y Z4 en E4 y en E3 con B = 1; en la tarjeta eso equivale a encender los segmentos a y d del HEX0. En la Figura 11 también se ve la diferencia entre los dos tipos de salida: Z2 y Z4 cambian en el instante en que B cambia (alrededor de 460 ns), antes del pulso de reloj; en la tarjeta pasa lo mismo al mover SW1 estando en E3. Esto ocurre porque son salidas condicionales que dependen de la entrada; Z1 y Z3 solo cambian en los flancos de bajada de KEY0 porque dependen únicamente del estado.

Conclusiones

Rea Alberto Cristian. Esta práctica me dejó claro que la carta ASM, la tabla y las ecuaciones son la misma máquina vista de tres maneras, y que un error en la primera se arrastra a todo lo demás. Lo que más cuidado pidió fue pasar de la carta a la tabla, porque las salidas condicionales de E3 hacen que Z2 y Z4 dependan también de B y por eso sus mapas ya no son solo de estado. Con seis variables los mapas de 8 × 8 son largos, pero las X de los códigos 101, 110 y 111 ayudaron bastante: D2 quedó en dos términos y Z1 en uno solo. Al revisar las 40 filas válidas contra las ecuaciones simplificadas coincidieron todas, y eso me dio confianza antes de pasar al VHDL. Me llevo también que conviene fijar la convención de las salidas negadas desde el principio; si no, es fácil invertir Z3 o Z4 al llenar la tabla.

Ortega de la Paz Rafael. Lo más útil para mí fue ver cómo las ecuaciones de los mapas pasan casi directo a VHDL: cada D y cada salida quedó como una asignación concurrente y un solo proceso guarda los tres flip-flops, con reset asíncrono en KEY1 y reloj manual en KEY0. Usar el botón como reloj permite revisar cada transición con calma, cosa que con el reloj de 50 MHz no se vería. Las salidas con lógica negada me hicieron pensar en cómo se conectan a la tarjeta: al

llevarlas a los segmentos del HEX0, que prenden con 0, se entiende que "activa en cero" no es un detalle del código sino de cómo está hecho el hardware. Comparado con la Práctica 1, donde solo conectamos un 74193, aquí aprendí que diseñar la lógica es lo que más tiempo lleva y que Quartus solo implementa lo que uno le describe.

Referencias

Mano, M. M., & Ciletti, M. D. (2018). Digital design: With an introduction to the Verilog HDL, VHDL, and SystemVerilog (6.ª ed.). Pearson.

Savage Carmona, J., Vázquez, G., & Chávez Rodríguez, N. E. (2017). Diseño de microprocesadores. Facultad de Ingeniería, Universidad Nacional Autónoma de México.

Terasic. (s. f.). DE10-Lite user manual. Terasic Technologies.

Wakerly, J. F. (2018). Digital design: Principles and practices (5.ª ed.). Pearson.